Task 4 – Stage, Commit & View History

Git & GitHub Fundamentals (Industry Workflow) | Estimated Time: 1 Hour

---

Note: The command block in the task page has two lines merged together. The corrected, separated commands are shown below in each step.

Step 1 – Stage All Files

Add all files in the current directory to the staging area (index):

```
git add .
```

The dot `.` means 'everything in this directory and below'. You can also stage individual files with `git add README.md`. Staging is a preparation step — it does not save a commit yet.

Step 2 – Verify the Staging Area

Confirm the files moved from 'untracked' to 'staged':

```
git status
```

All three files should now appear in green under Changes to be committed. If any file is still red it was not staged — re-run `git add .` and check again.

Step 3 – Create the Initial Commit

Save a permanent snapshot of the staged files with a commit message (these are two separate commands):

```
git commit -m "Initial Commit"
```

```
git log
```

`git commit -m` saves the snapshot. The `-m` flag lets you write the message inline. `git log` then displays the full commit history — you should see your commit with its hash, author, date, and message.

Step 4 – View a Compact Log

See a condensed one-line summary of each commit — useful once you have many commits:

```
git log --oneline
```

Each line shows the short commit hash (7 characters) and the commit message. Also try `git show` to display the full details and diff of the most recent commit.

```
git show
```

Step 5 – Modify a File and Commit Again

Edit a file, stage the change, and commit — repeat this cycle to build up at least 5 commits:

```
echo "Added Git notes" >> notes.txt
git add notes.txt
git commit -m "Add Git learning notes to notes.txt"
```

Use >> (double arrow) to append to a file without overwriting it. > (single arrow) would replace the file's contents.

Plan: 5 Meaningful Commits

The deliverable requires a minimum of 5 commits with meaningful messages. Here is a suggested sequence — run each edit + add + commit block one at a time:

Commit 1 — Initial Commit

```
git add .
git commit -m "Initial Commit: add README, notes, commands files"
```

Commit 2 — Update README

```
echo "## About\nThis repo tracks my DevOps Git learning." >> README.md
git add README.md
git commit -m "Update README with project description"
```

Commit 3 — Add Git notes

```
echo "git init - initializes a repo" >> notes.txt
git add notes.txt
git commit -m "Add git init explanation to notes.txt"
```

Commit 4 — Log commands

```
echo "git add . | git commit -m | git log" >> commands.txt
git add commands.txt
git commit -m "Log stage and commit commands in commands.txt"
```

Commit 5 — Final update

```
echo "git log --oneline | git show" >> commands.txt
git add commands.txt
git commit -m "Add log viewing commands to commands.txt"
```

After all 5 commits, verify the full history with:

```
git log --oneline
```

Key Concepts

- **Working directory** — Where you edit files — changes here are not tracked until staged.
- **Staging area (index)** — A preparation zone. `git add` moves files here.
- **Commit** — A permanent snapshot saved in the repository. Cannot be lost once committed.

• **Commit hash** — A unique 40-character ID (shown as 7 chars in --oneline) identifying each commit.

• **HEAD** — A pointer to the most recent commit on the current branch.

• **git log** — Shows full commit history: hash, author, date, message.

• **git log --oneline** — Compact view — one line per commit with short hash and message.

• **git show** — Displays full details and the diff (line-by-line changes) of the latest commit.